

好，正式进入。

这一步只做**架构方案冻结**，不写代码，不接 OCR，不动现有 V3.5。

《LoveAdvisor V3.6 架构方案》

—— 图转文输入节点引入阶段（纪律严格版）

一、阶段判断

1.1 为什么必须单独开 V3.6

V3.5 已经完成了以下核心能力：

- * 文本输入路径可用
- * 截图上传路径可用
- * 输入确认层可用
- * 分析引擎可用
- * 结果页产品化可用
- * MVP 试点版可运行

也就是说，V3.5 的定位已经成立：

``text id="hmt9mv"

V3.5 = 试点级可用产品版本

``

但当前截图路径仍然存在一个明确限制：

``text id="cxzvw1"

截图 → 模拟转写文本 → 用户确认 → 分析

``

这里的“模拟转写文本”只是占位实现，不具备真实图像转文字能力。

因此，如果要继续推进“截图真实可用”，就不是补一个小功能，而是要**在输入层正式引入一个新节点**：

```text id="qvrqh5"

I1: 图转文节点

```

这会改变输入层链路结构，所以必须单独立项，不能按临时补丁施工。

1.2 V3.6 的阶段名称

建议正式命名为：

```text id="dyep0n"

LoveAdvisor V3.6: 图转文输入节点引入阶段

```

这个命名强调的是：

- * 不是“临时接 OCR”
- * 不是“顺手补功能”
- * 而是**输入层架构升级**

二、V3.6 的核心目标

V3.6 只做一件大事：

```text id="ke5s5u"

把截图输入从“模拟转写”升级为“真实图转文节点”

```

具体拆开有三个子目标：

G1: 新增 I1 图转文节点

让截图输入具备真实文字提取能力。

G2: 保持 I2 确认层不变

图转文结果必须继续进入用户确认层，不能直通分析引擎。

G3: 不破坏现有 V3.5 主链路

文本输入路径、分析引擎、结果页都视为成熟资产，默认不重构。

三、核心原则

3.1 原则一：图转文是节点，不是小功能

必须明确：

```
```text id="0i0qi0"
```

图转文 = 新输入节点 = 新架构要素

```
```
```

所以它需要：

- * 明确职责
- * 明确边界
- * 明确输入输出协议
- * 明确与 I2 的衔接方式

不能把它当作前端里的一段“顺手调用识别接口”的代码。

3.2 原则二：I1 不得替代 I2

I1 图转文节点只负责：

- * 接收图片
- * 识别文字
- * 合并多图文本
- * 返回转写初稿

I1 明确**不负责**：

- * 关系分析
- * 用户问题理解
- * 说话人最终纠正
- * 跳过确认层直接分析

必须强制保留这条规则：

```
```text id="3x0t3k"
```

所有图转文结果，必须先进入 I2 确认层，禁止直接进入分析引擎

```
```
```

3.3 原则三：分析引擎保持黑箱稳定

V3.6 默认不改：

- * `/api/v1/analyze`
- * `rules.py`
- * `pipeline\_orchestrator`
- * S2 / S3 / R1 / S5
- * 结果页主展示逻辑

也就是说，图转文接入的正确方式是：

```
```text id="8bpl2v"
```

I1 输出文本

→ I2 用户确认

→ confirmed\_chat\_text

→ 继续复用现有 analyze

```
```
```

而不是把图转文塞进 analyze 内部。

3.4 原则四：图转文能力与分析能力解耦

必须把两个能力拆开：

能力 A：图像转文字

输入：图片

输出：文本初稿

能力 B：关系分析

输入：聊天文本

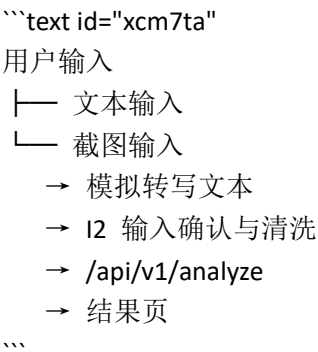
输出：分析结果

两者必须独立存在，不能混成一个接口，不然以后：

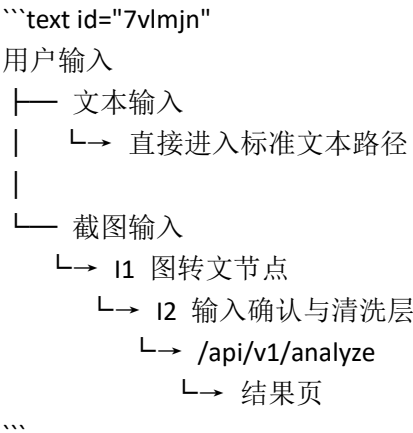
- * 调试困难
- * 替换困难
- * 成本评估困难
- * 错误定位困难

四、V3.6 新架构

4.1 旧链路（V3.5）



4.2 新链路（V3.6）



4.3 V3.6 的输入层结构

```text id="r4krxj"

输入层

- └─ T1 文本输入路径
- └─ I1 图片转文字路径
  - └─ I2 用户确认与清洗

```

这里最关键的是：

```text id="rl3n9m"

- I1 负责“自动生成草稿”
- I2 负责“用户确认后定稿”

```

五、I1 图转文节点设计

5.1 I1 的职责

I1 只负责以下事情：

1. 接收前端上传的单张或多张截图
2. 调用图转文能力（OCR 或视觉模型）
3. 提取原始文字内容
4. 合并多张截图的识别结果
5. 输出文本初稿给 I2

5.2 I1 不负责什么

I1 明确不做：

- * 最终说话人校正
- * 对话轮次整理到最优格式
- * 关系判断
- * 建议生成
- * 用户提问理解
- * 直接触发分析

5.3 I1 的边界定义

一句话定义:

```
```text id="qlij31"
I1 = 自动草稿生成器，不是分析器，不是最终文本决定者
```
```

六、I1 输出协议

6.1 最低标准输出协议

建议 I1 输出统一为:

```
```json id="ugfsa2"
{
 "success": true,
 "raw_text": "识别得到的原始文本",
 "merged_text": "合并后的聊天文本初稿",
 "provider": "vision_model_or_ocr",
 "source_type": "image",
 "image_count": 2,
 "need_user_confirm": true,
 "error_message": ""
}
```
```

6.2 字段含义

| 字段 | 含义 |
|-------------|--------------|
| success | 本次图转文是否成功 |
| raw_text | 原始识别文本 |
| merged_text | 面向 I2 的可编辑初稿 |
| provider | 当前使用的图转文实现来源 |

| | | |
|-------------------|------------------|--|
| source_type | 输入类型，固定为 image | |
| image_count | 本次上传图片数量 | |
| need_user_confirm | 是否必须用户确认，固定 true | |
| error_message | 失败时返回错误描述 | |

6.3 为什么必须有 raw_text 和 merged_text 双字段

因为后续会有两种调试需求：

- * 看“识别得对不对”
- * 看“整理后的草稿好不好”

如果只保留一个字段，后面定位会非常困难。

七、I2 输入确认层在 V3.6 中的定位

7.1 I2 继续保留

V3.6 不新增“跳过确认”的快捷路径。

必须保持：

```
```text id="pnfxyi"
I1 输出 → I2 确认 → confirmed_chat_text → analyze
```
```

7.2 I2 的职责继续不变

I2 负责：

- * 展示图转文草稿
- * 允许用户编辑
- * 允许用户确认
- * 产出 `confirmed\_chat\_text`

I2 不负责：

- * 真正识图
- * 后端模型调用
- * 分析推理

八、技术路线选择框架

这一步先不定死实现，只冻结架构上的选择框架。

8.1 候选路线 A：云端视觉模型读图

特点：

- * 开发快
- * 对复杂聊天截图更灵活
- * 适合试点验证
- * 成本相对更高
- * 依赖外部模型服务

适合阶段：

``text id="zsiblr"

优先用于快速验证 V3.6

``

8.2 候选路线 B：传统 OCR / 本地 OCR

特点：

- * 更像纯识字
- * 成本可能更低
- * 本地可控性更高
- * 对复杂聊天布局可能不如多模态模型灵活

适合阶段：

``text id="6kz4v6"

用于后续降本或本地化增强

```

---

### ## 8.3 架构层结论

V3.6 阶段只冻结：

``text id="ki4dxy"

I1 是独立节点

```

暂时不在架构文档中绑定某个唯一实现。

也就是说：

- * 先定“节点”
- * 后定“节点内部怎么做”

这样纪律最稳。

九、V3.6 分阶段实施建议

Phase 1：I1 架构与协议冻结

目标：

- * 完成图转文节点方案文档
- * 冻结 I1 / I2 边界
- * 冻结 I1 输出协议

产出：

- * 架构文档
- * 数据协议
- * 职责说明

边界：

- * 不写代码

Phase 2: 后端 I1 接口实现

目标:

- * 新增独立图转文接口
- * 接收图片并返回 I1 输出协议

建议接口:

```
``text id="uxjlwm"
POST /api/v1/image-to-text
``
```

或

```
``text id="oqpyjq"
POST /api/v1/transcribe-image
``
```

边界:

- * 不改 analyze
- * 不改 rules
- * 不改 pipeline

Phase 3: 前端接入 I1

目标:

- * 截图上传后不再生成模拟文本
- * 改为调用 I1 接口获得真实转写草稿
- * 回填 I2 确认框

边界:

- * 不改结果页
- * 不改 analyze 逻辑

Phase 4: 错误处理与联调

目标:

- * 处理图转文失败
- * 处理空结果
- * 处理多图顺序合并
- * 衔接 I2 → analyze

边界:

- * 不做额外功能扩展

Phase 5: 评测与试点回归

目标:

- * 验证真实截图路径是否可用
- * 比较文本输入与截图输入路径体验
- * 记录图转文准确率与确认成本

十、接口边界建议

10.1 新接口必须独立

建议新增:

```
``text id="vjlv2r"
POST /api/v1/image-to-text
``
```

而不是复用:

```
``text id="15vj4d"
/api/v1/analyze
``
```

因为两者语义不同：

| | | |
|---------------|---------|--|
| 接口 | 职责 | |
| ----- | ----- | |
| image-to-text | 图片转文本 | |
| analyze | 文本转分析结果 | |

10.2 为什么不能合并

如果合并，会出现：

- * 输入协议混乱
- * 错误定位困难
- * 后续替换 I1 实现困难
- * 前端流程被迫耦合

所以必须坚持：

```text id="fa0n0c"  
图转文接口 ≠ 分析接口  
```

十一、风险与控制策略

风险 1：把 I1 做成“自动最终版文本”

控制策略：

- * 强制保留 I2
- * 所有图转文结果必须用户确认

风险 2：顺手改 analyze

控制策略：

- * analyze 完全冻结
- * 新能力只走独立接口

风险 3：Claude Code 借 I1 扩散到大重构

控制策略：

- * 严格 MTU
- * 每次只做一个模块
- * 每次只改指定文件

风险 4：前端和后端协议不一致

控制策略：

- * 先冻结 I1 输出协议
- * 再开发接口和前端接入

十二、阶段结论

V3.6 不是“接 OCR”这么简单，而是一次正式的输入层架构升级。

它的本质是：

``text id="oerxfx"

把截图输入从“占位演示”升级为“真实图转文节点”

``

因此，正确的做法不是直接施工，而是先把 I1 节点纳入架构体系，再按纪律分阶段推进。

十三、当前结论

现在应正式执行：

``text id="q4jry2"

冻结 V3.5

启动 V3.6

先做 I1 节点方案

再做接口

再做前端接入

``

十四、下一步口令

如果你认可这份方案，下一句直接说：

```
```text id="g7a9ty"
```

开始 V3.6 Phase 1

```
```
```

我就给你一张**严格 MTU 的 Claude Code 任务单**，只做：

```
```text id="zso8eo"
```

把 V3.6 图转文节点方案落成正式文档

```
```
```